Requirement ID	Description	Story Points	Priority	Sprint No.	Notes:
1	Display start screen that prompts the user to play	5	1	1	Base template
2	Create window and main game loop	5	2	1	
3	Load word list from a local txt file	3	3	1	
4	Select a random target word of a given length	2	4	1	
5	Draw game board grid with 5 guess rows	8	5	1	
6	Prompt the user to enter a guess	1	6	1	User input
6.1	Display typed letters in the current row as the user types	5	7	1	
6.2	Support backspace to delete the last typed letter	3	8	1	
6.3	Submit a guess with the Enter key	2	9	1	
6.4	Reject incomplete guesses with an error message	2	10	1	
7	Evaluate a submitted guess against the target word	5	11	1	Matching
7.1	Color tiles green for correct letter in correct position	3	12	1	
7.2	Color tiles yellow for correct letter in wrong position	3	13	1	
7.3	Color tiles grey for letters not in the word	1	14	1	
8	Detect a win when the guess matches the target word	2	15	1	Win screen
8.1	Display win screen after a correct guess	3	16	1	
8.2	Offer retry option on the win screen	2	17	1	
9	Detect a loss when all guesses are used	2	18	1	Lose screen
9.1	Display lose screen that reveals the target word	3	19	1	
9.2	Offer retry option on the lose screen	2	20	1	
10	Reset game state and pick a new word on retry	3	21	1	Main loop sequence
11	Add difficulty selection screen	3	22	2	Difficulty feature
11.1	Prompt the user to select a difficulty (EASY, MEDIUM, HARD), using buttons or pressing `e`, `m`, `h` keys. (Should happen before the game screen begins)	3	23	2	
11.2	Build separate word lists for each word length (EASY = 3 letters, MEDIUM = 5 letters, Hard = 6 letters) (Lists can be made in different word_dictionary.txt files)	2	24	2	
11.3	Adjust the word_handler to get the words of a certain length/difficulty list	3	25	2	
11.4	Adjust the grid UI to match the word length	5	26	2	
12	Create score calculation system	2	27	2	User score
12.1	Award points for each correctly placed letter	2	28	2	
12.2	Award bonus points for winning in fewer guesses	2	29	2	
12.3	Display the current score on the game screen, win screen, and lose screen	3	30	2	
13	Add vowel hint button on the game screen	3	31	2	Hint feature
13.1	Display an extra row at the bottom of the screen showing the location of the hint letters. (ex. chosen word is apple; Hint would display [A][ ][ ][ ][E])	5	32	2	
13.2	Limit hint use to once per game	2	33	2	
13.3	Apply a score penalty when the hint is used	2	34	2	
14	Record the results of each completed game (i.e. at win screen)	1	35	2	User stats
14.1	Prompt the user to enter their player name (Names can be short; three letters, `AAA` `ABC` `JOE` `AXL` `EXE` and etc.)	3	36	2	
14.2	Record the number of guesses used	2	37	2	
14.3	Record the score earned in game	2	38	2	
14.4	Record the difficulty level played in each game	2	39	2	
14.5	Save game records to a local file so they persist between sessions. (Store in a .txt file and add each record in a format for each line) (Format: <name>,<numberOfGuesses>, <score>,<difficulty>) If the name already exist relpace that entry in the .txt file	5	40	2	
15	Create leaderboard ranking games by score	5	41	2	Leader Board
15.1	Sort the game records in the local file or during program runtime.	3	42	2	
15.2	Break score ties using the fewest guesses	2	43	2	
15.3	Filter by the current game difficulty and select the top 5 entries and current player	3	44	2	
15.4	Add a button to the win screen to toggle the leaderboard	5	45	2	
15.5	Build leaderboard screen showing player names, scores, guesses, and difficulty	5	46	2	
15.6	List top 5 players and at the bottom the current player	3	47	2	
16	Create VS mode where two players alternate guesses	3	48		VS mode
16.1	Prompt both players to enter their names in VS mode	3	49		
16.2	Alternate turns between players in VS mode	5	50		
16.3	Show whose turn it is on the VS mode screen	3	51		
16.4	Declare the first player to guess the word the VS mode winner	3	52		